

AI-Based Smart Complaint Management System

MERN Stack Examination Project Report

Course	AI Driven Full Stack Development (AI308B)
Case Study	AI-Based Smart Complaint Management System
Prepared For	B.Tech 4th Semester ESE 2025-26

This project implements a full MERN Stack complaint portal where citizens can register and track complaints online. The system uses AI-assisted analysis to classify complaint priority, summarize the issue, recommend the concerned department, and generate an automated response message.

1. Objectives

- Build a secure MERN Stack application for complaint registration and tracking.
- Apply AI analysis for urgency detection, department recommendation, summary generation, and automated responses.
- Implement JWT authentication, protected routes, and bcrypt password hashing.
- Prepare the application for GitHub upload and Render deployment.

2. Frontend Modules

Module	Description
Complaint Registration Form	Captures user name, email, title, description, category, location, and status.
Complaint List Page	Displays all complaints with category filter, text search, and AI result cards.
Complaint Status Update Page	Allows users and admins to update complaint status values.
AI Analysis Result Display	Shows priority, department suggestion, summary, auto-response, and AI source.

3. Backend APIs

Endpoint	Purpose
POST /api/auth/signup	Register a new user or admin and generate JWT token.
POST /api/auth/login	Authenticate user and return signed JWT token.
GET /api/auth/me	Fetch logged-in user profile using protected route.
POST /api/complaints	Store a new complaint with AI analysis fields.
GET /api/complaints	Fetch complaints with filter and search support.
PUT /api/complaints/:id	Update complaint status.
DELETE /api/complaints/:id	Delete complaint record.
GET /api/complaints/search?location=Ghaziabad	Search complaints by location.
POST /api/ai/analyze	Generate AI analysis without storing the complaint.

4. MongoDB Schema

The complaint collection stores base complaint details along with AI analysis output. Additional metadata includes creator reference and timestamps for auditability.

Field	Type / Purpose
name, email, title, description, category, location	Core complaint input fields with validation.
status	Pending, In Progress, Resolved, or Rejected.
priority	AI-detected urgency level: Low, Medium, High, or Critical.
department	Suggested department such as Water Department or Sanitation Department.
summary	Short AI-generated summary of the complaint.
autoResponse	Automated message shown to the complainant.
createdBy	MongoDB ObjectId reference to the authenticated user.
timestamps	Automatically stores created and updated dates.

5. Authentication and Security

- Passwords are hashed using bcrypt before storage.
- JWT tokens are generated during signup and login.
- Protected middleware blocks access to complaint APIs without valid token.
- Input validation is applied using express-validator.

6. AI Integration

The application supports OpenRouter API integration for intelligent analysis. To keep the project functional during exams or network issues, a heuristic fallback engine also classifies complaints and generates meaningful outputs.

Complaint Type	Expected AI Output
Water leakage	High priority with Water Department recommendation.
Electricity issue	High or Critical priority with Electricity Department recommendation.
Garbage complaint	Sanitation Department recommendation.
Long complaint text	Concise summary and response message.

7. Manual Test Results

Test Case	Observed Result
User signup	Signup successful and token generated.
User login	Login successful and token generated.
Add complaint	Complaint stored successfully.
AI routing	Water complaint mapped to Water Department and High priority.
Update complaint status	Status updated to Resolved.
Invalid input	Validation returned 400 Bad Request.

8. Deployment Readiness

- Render blueprint provided in render.yaml for frontend and backend services.
- Environment templates added for MongoDB URI, JWT secret, client URL, and OpenRouter settings.
- Root-level scripts added for development, build, and backend start.

9. Screenshots to Insert Before Submission

- Signup and login page screenshots
- Complaint registration form screenshot
- Complaint list and status update screenshot
- AI analysis result screenshot
- Postman or Thunder Client request screenshots
- MongoDB Atlas collection screenshot
- Render deployment and live endpoint screenshots

10. Conclusion

The Smart Complaint Management System demonstrates practical MERN integration with AI-assisted automation. It satisfies the examination requirements for frontend modules, backend APIs, MongoDB schema, authentication, AI features, code quality, and deployment preparation.

Appendix: Project Files

- README.md for setup, endpoints, and deployment guidance
- REPORT.md for the editable report source
- client/ React frontend source code
- server/ Express and MongoDB backend source code
- render.yaml for Render deployment